Tugas Besar 1
IF3270 Pembelajaran Mesin

FeedForward Neural Network

Disusun oleh:

Ivant Samuel Silaban	13523129
Rafa Abdussalam Danadyaksa	13523133
Muhammad Rizain Firdaus	13523164

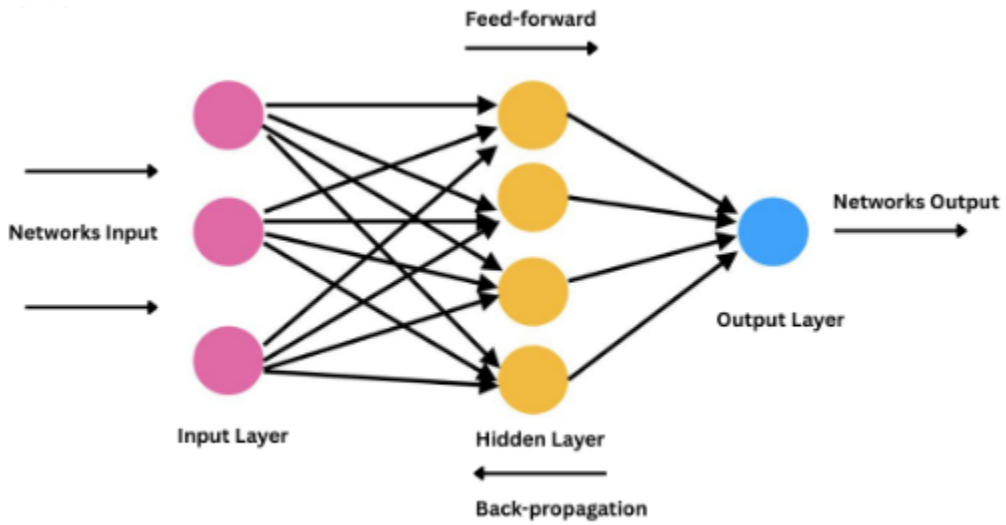
PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

DAFTAR ISI.....1
BAB I: Deskripsi Persoalan.....2
BAB II: Pembahasan.....3
2.1 Implementasi.....3
2.1.1 Deskripsi Kelas, Atribut, dan Metodenya.....4
2.1.2 Forward Propagation.....7
2.1.3 Backward Propagation dan Weight Update.....10
2.2 Hasil Pengujian.....16
2.2.1 Pengaruh Depth dan Width.....16
2.2.2 Pengaruh Fungsi Aktivasi.....21
2.2.3 Pengaruh Learning Rate.....26
2.2.4 Pengaruh Inisialisasi Bobot.....29
2.2.5 Pengaruh Regularisasi.....34
2.2.6 Perbandingan dengan Library Scikit-Learn.....38
2.2.7 Perbandingan dengan Adam.....40
BAB III: Kesimpulan dan Saran.....44
3.1 Kesimpulan.....44
3.2 Saran.....45
LAMPIRAN.....46
REFERENSI.....47
Lampiran Form Penggunaan AI.....48

BAB I: Deskripsi Persoalan



Gambar 1.1 Feedforward Neural Network

Tugas Besar I pada kuliah IF3270 Pembelajaran Mesin agar peserta kuliah mendapatkan wawasan tentang bagaimana cara mengimplementasikan Feedforward Neural Network (FFNN). Pada tugas ini, peserta kuliah akan ditugaskan untuk mengimplementasikan FFNN from scratch.

Persoalan utama dalam tugas ini adalah bagaimana peserta dapat membangun arsitektur jaringan saraf (*neural network*) yang fleksibel dalam menentukan neuron serta *layer* yang dibutuhkan, implementasi proses *forward* dan *backward propagation* untuk proses training neural network, penerapan fungsi aktivasi dan fungsi loss serta mekanisme inisialisasi bobot yang mendukung proses training. Model yang sudah dibangun kemudian diterapkan pada dataset yang disediakan di spesifikasi dan peserta juga diminta untuk melakukan analisis pengaruh hyperparameter seperti learning rate, regularisasi, hingga perbandingan performa dengan library Sklearn MLP.

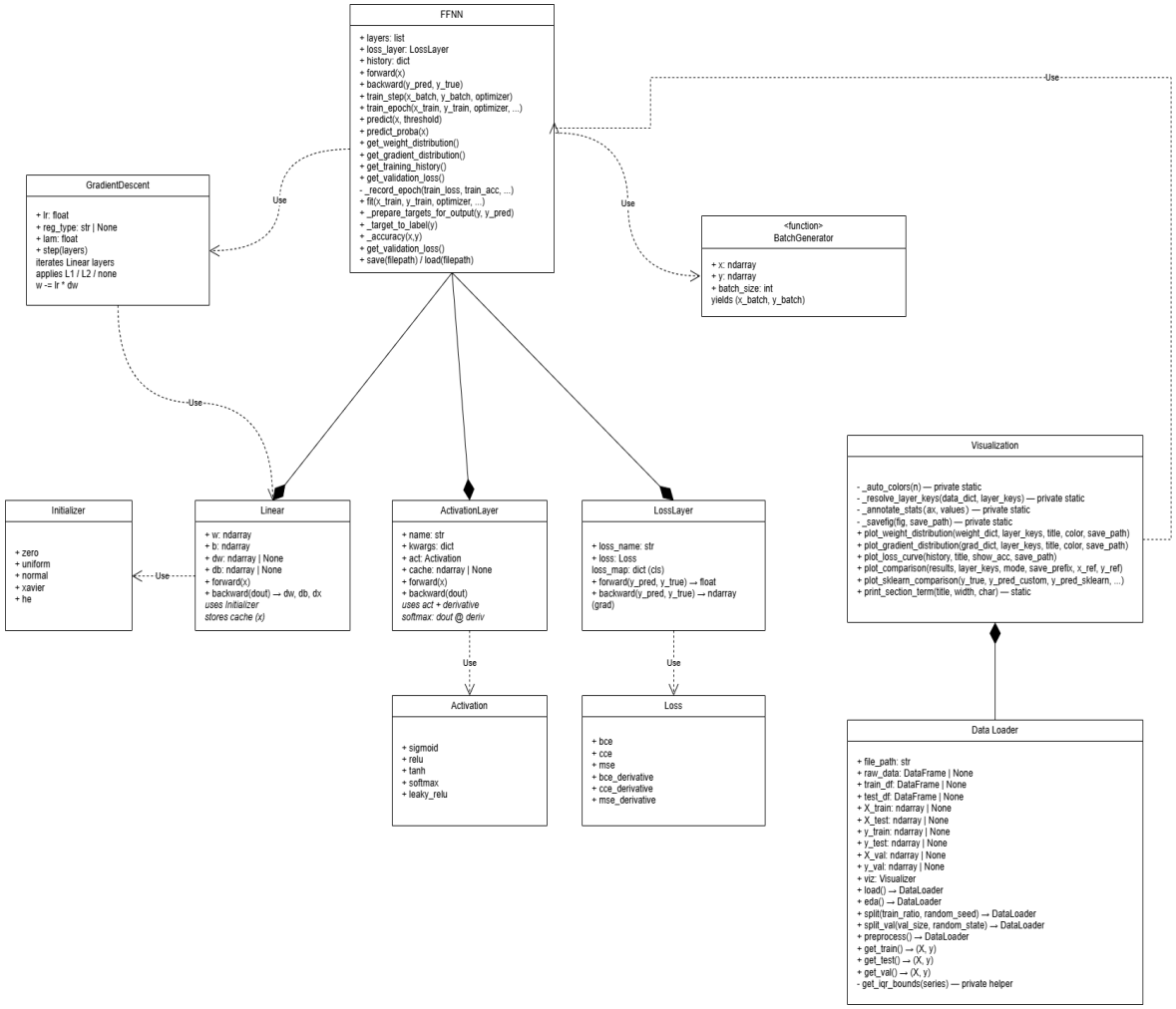
BAB II: Pembahasan

2.1 Implementasi

Tabel 2.1.1 Struktur direktori program

```
├── README.md
├── data (dataset)
│   └── datasetml_2026.csv
├── pyproject.toml
├── report (laporan)
├── results (hasil model/output model)
├── src
│   ├── __init__.py
│   ├── __pycache__
│   ├── main.py
│   ├── model
│   └── utils
├── tests (kumpulan notebook analisis)
│   ├── activation_analysis.ipynb
│   ├── depth_width_analysis.ipynb
│   ├── learning_rate_analysis.ipynb
│   ├── regularization_analysis.ipynb
│   ├── sklearn_comparison.ipynb
│   ├── testing.ipynb
│   └── weight_init_analyzer.ipynb
└── uv.lock
```

2.1.1 Deskripsi Kelas, Atribut, dan Metodenya



Gambar 2.1.1.1 Diagram Kelas, Atribut, dan Metode

Sistem ini adalah implementasi **Feedforward Neural Network (FFNN) dari scratch** menggunakan Python dan NumPy, tanpa bergantung pada framework deep learning seperti PyTorch atau TensorFlow. Arsitekturnya dirancang modular setiap tanggung jawab dipisahkan ke kelas yang berbeda, sehingga mudah dipahami, diuji, dan dikembangkan secara independen. **Inti sistem** ada di kelas FFNN yang berperan sebagai orkestrator utama. FFNN tidak mengimplementasikan operasi matematis

sendiri secara langsung ia mendelegasikan setiap pekerjaan ke kelas-kelas yang lebih spesifik. Ketika FFNN dibuat, ia secara otomatis membangun arsitektur jaringan dengan menyusun layer-layer secara berurutan: setiap hidden layer terdiri dari sepasang Linear dan *ActivationLayer*, diakhiri dengan Linear dan *ActivationLayer* untuk output. Selain itu FFNN memiliki satu *LossLayer* untuk menghitung error dan gradiennya. Karena ketiga kelas ini dibuat dan dimiliki sepenuhnya oleh FFNN, hubungannya adalah ***composition*** mereka tidak bisa berdiri sendiri tanpa FFNN.

Alur komputasi berjalan dua arah. Saat forward pass, data mengalir dari input melalui setiap Linear dan Activation Layer secara berurutan hingga menghasilkan prediksi. Setiap layer menyimpan *cache* input-nya masing-masing untuk keperluan backward pass. Saat backward pass, *LossLayer* menghitung gradien loss terhadap prediksi, lalu gradien tersebut mengalir balik melewati setiap layer dalam urutan terbalik *ActivationLayer* mengalikan gradien dengan turunan fungsi aktivasi, lalu Linear menghitung gradien terhadap bobot (*dw*), bias (*db*), dan input (*dx*). Setelah backward pass selesai, *GradientDescent* masuk sebagai *optimizer* yang mengambil semua layer, mencari yang memiliki bobot, lalu memperbarui nilainya. Hubungan FFNN ke *GradientDescent* adalah **dependency** karena *optimizer* diterima sebagai parameter dari luar, bukan dibuat sendiri oleh FFNN ini memungkinkan *optimizer* diganti dengan mudah tanpa mengubah kode FFNN. FFNN menyediakan method fit sebagai training loop yang akan menerima parameter saat dipanggil. Untuk prediksi, method predict mendukung dua mode yaitu binary classification dan multiclass classification dengan argmax. **Layer-layer internal** masing-masing memiliki ketergantungan ke modul eksternal. Linear bergantung ke *Initializer* hanya saat konstruksi untuk menginisialisasi bobot dengan berbagai strategi seperti *zero*, *uniform*, *normal, Xavier,* atau *He* setelah itu *Initializer* tidak dibutuhkan lagi. *ActivationLayer* bergantung ke modul *Activation* yang menyimpan implementasi aktual fungsi-fungsi aktivasi dan turunannya. *LossLayer* bergantung ke modul *Loss* yang menyimpan implementasi *BCE, CCE,* dan *MSE* beserta turunannya. Ketiga modul eksternal ini (*Initializer,*

Activation, Loss) adalah bagian dari sistem tapi dipisahkan agar masing-masing bisa dikembangkan atau diganti secara independen.

Proses training juga melibatkan *BatchGenerator* sebagai fungsi standalone yang dipanggil oleh FFNN di setiap epoch. *BatchGenerator* bertugas memecah dataset besar menjadi *batch-batch* kecil secara acak, sehingga training bisa dilakukan dengan mini-batch gradient descent. Karena ini hanya sebuah fungsi biasa bukan kelas, ia tidak memiliki state dan hubungannya ke FFNN adalah **dependency** sederhana. **Di luar sistem *neural network* itu sendiri**, terdapat dua kelas pendukung. *DataLoader* mengurus seluruh pipeline data mulai dari membaca file CSV, eksplorasi data, *splitting* train/test/validation, preprocessing berupa *clipping outlier, encoding* kategorikal, hingga normalisasi fitur numerik. *DataLoader* memiliki *Visualizer* sebagai bagian dari dirinya sendiri melalui ***composition***, menggunakannya untuk menampilkan informasi selama proses EDA dan *pre-processing*. *Visualizer* sendiri adalah kelas utilitas murni yang tidak menyimpan state apapun ia hanya menerima data dan menghasilkan plot. Yang menarik, *Visualizer* juga memiliki ***dependency*** ke FFNN karena mampu memvisualisasikan distribusi bobot dan gradien dengan cara memanggil langsung *method-method* milik FFNN. Ini berarti *Visualizer* bisa digunakan baik oleh *DataLoader* untuk keperluan EDA, maupun secara langsung oleh pengguna untuk menganalisis kondisi internal model setelah training.

Secara keseluruhan, class diagram ini menggambarkan sistem yang **bersih dan terpisah secara tanggung jawab**. Tidak ada *inheritance* sama sekali karena setiap kelas memang memiliki peran yang unik dan tidak saling berbagi kontrak. Pola dominan yang digunakan adalah ***composition*** untuk kepemilikan objek dan ***dependency*** untuk kolaborasi antar kelas tanpa kepemilikan sebuah pendekatan yang membuat sistem ini fleksibel, mudah diuji per bagian, dan terbuka untuk pengembangan seperti menambahkan *optimizer* baru, fungsi aktivasi baru, atau strategi inisialisasi baru tanpa harus mengubah kelas-kelas yang sudah ada.

2.1.2 Forward Propagation

Forward propagation adalah proses mengalirkan data input dari layer pertama hingga ke layer output untuk menghasilkan prediksi. Setiap layer menerima output layer sebelumnya, melakukan transformasi, lalu meneruskan hasilnya ke layer berikutnya.

Model FFNN yang diimplementasikan terdiri dari rangkaian layer yang disusun secara berurutan di dalam self.layers.

```
dims = [input_dim] + hidden_dim

for i in range(len(dims) - 1):
    self.layers.append(Linear(dims[i], dims[i + 1], **init_kwargs))
    self.layers.append(ActivationLayer(hidden_activation, **act_kwargs))

self.layers.append(Linear(dims[-1], output_dim, **init_kwargs))
self.layers.append(ActivationLayer(output_activation))
```

Jadi jika parameter hidden_dim adalah [256, 128, 64], maka arsitekturnya adalah 21 -> 256 -> 128 -> 64 -> 1, dan self.layers akan berisi:

```
[Linear(21->256), ActivationLayer(relu), Linear(256->128),
ActivationLayer(relu), Linear(128->64),
ActivationLayer(relu), Linear(64->1),
ActivationLayer(sigmoid)]
```

Data input akan masuk ke network. Jika kita memiliki input x, maka input ini akan diteruskan ke hidden layer pertama. Di setiap neuron pada hidden layer, akan terjadi dua proses utama:

1. Operasi Linear:

   Input akan dikalikan dengan bobot (w) dan ditambah dengan bias(b).

   $$net_z = \sum(x_i \cdot w_i) + b$$

   Atau jika dalam bentuk matriks:

   $$net_Z = X \cdot W + B$$

2. Fungsi Aktivasi

Hasil z tersebut kemudian dimasukkan ke dalam fungsi aktivasi untuk memberikan sifat non-linear pada model. Tanpa fungsi aktivasi, FFNN hanya akan berfungsi seperti regresi linear biasa.

$$z = activation(net_z)$$

Dengan fungsi aktivasi sebagai berikut:

Fungsi Aktivasi	**Formula dan Implementasi**
Linear	$Linear(x) = x$
	`def linear(self, x):` `  return x`
Sigmoid	$\sigma(x) = \frac{1}{1 + e^{-x}}$
	`def sigmoid(self, x):` `  return 1/ (1 + np.exp(-x))`
ReLU	$ReLU(x) = \max(0, x)$
	`def relu(self, x):` `  return np.maximum(0, x)`
Hyperbolic Tangent	$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$

	`def tanh(self, x):` `    return np.tanh(x)`
Softmax	Untuk vector $\vec{x} = (x_1, x_2, \dots, x_n) \in \mathbb{R}^n$, $softmax(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$

	`def softmax(self, x):`
	`    exp_x = np.exp(x - np.max(x))`
	`    return exp_x / np.sum(exp_x)`

Leaky ReLU	$f(x) = \begin{cases} x & x > 0, \\ \alpha x & x \leq 0, \end{cases}$
	`def leaky_relu(self, x, alpha=0.01):` `    return np.where(x > 0, x, alpha * x)`
Softplus	$f(x) = \ln(1 + e^x)$
	`def softplus(self, x):` `    return np.log(1 + np.exp(x))`

Hasil aktivasi akan dikirim ke layer berikutnya sebagai input baru hingga akan menghasilkan prediksi dihasilkan di layer paling ujung. Tetapi tidak sampai segitu saja. Hasil prediksi di awal-awal biasanya tidak begitu bagus, dapat dilihat dari perhitungan akurasi dan fungsi loss yang merupakan fungsi untuk mengukur seberapa jauh perbedaan antar prediksi model dengan target asli atau label yang sebenarnya. Oleh sebab itu, kita membutuhkan backward propagation untuk memperbaiki hasil prediksi.

Implementasi FFNN.forward adalah sebagai berikut:

```python
def forward(self, x: np.ndarray) -> np.ndarray:
    out = x
    for layer in self.layers:
        out = layer.forward(out)
    return out
```

Iterasi sederhana melalui semua layer secara berurutan. Setiap layer.forward(out) melakukan satu transformasi dan meneruskan hasilnya.

2.1.3 Backward Propagation dan Weight Update

Backward Propagation (backprop) adalah algoritma untuk menghitung gradien loss terhadap setiap bobot di jaringan, dengan cara menelusuri alur komputasi dari output ke input menggunakan chain rule. Gradient ini kemudian digunakan oleh optimizer untuk memperbarui bobot.

Sesuai dengan paragraf terakhir untuk sub-bab 2.1.2, kita butuh backward propagation ini untuk memperbaiki hasil prediksi lewat meminimalkan loss $\mathcal{L}(\widehat{y}, y)$. Dan untuk meminimalkan loss, berarti kita memperbarui nilai bobot di tiap neuron nantinya. Tapi, bagaimana kita mengetahui seberapa besar pengaruh bobot W terhadap nilai loss tersebut? Kita akan menggunakan gradient $\frac{\partial \mathcal{L}}{\partial W}$.

Namun, bobot W tidak langsung terhubung ke loss, sehingga kita harus menggunakan Chain rule yang akan memecah persoalan ini dan mencari keterkaitan keduanya. Lalu setelah itu, bobot akan diperbarui berdasarkan seberapa besar “tanggung jawab” tiap bobot terhadap error.

Sebelum masuk ke backward pass, loss akan dihitung terlebih dahulu dari prediksi dan label. Loss function dapat berupa:

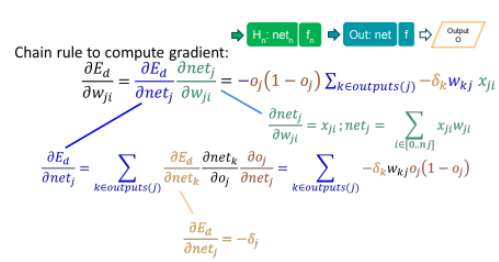
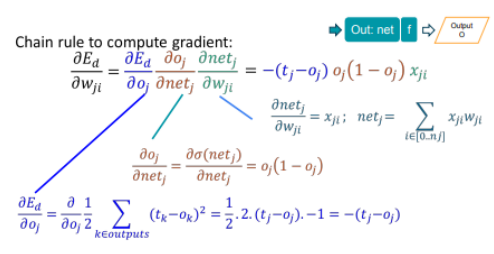
Nama Fungsi Loss	Formula dan Implementasi

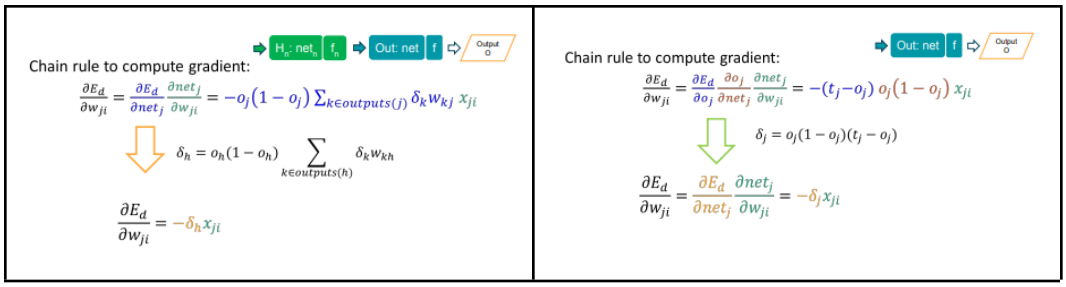
Mean Square Error (MSE)	$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$
	 `def mse(self, y_a, y_p, epsilon=1e-15):`
	 `  return np.mean((y_a - y_p)**2)`

Binary Cross-Entropy (BCE)	$$\mathcal{L}_{BCE} = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i))$$
	 y_i = Actual binary label (0 or 1)
	 $\hat{y}_i$ = Predicted value of y_i
	 n = Batch size
	 `def binary_cross_entropy(self, y_a,y_p, epsilon=1e-15):`
	 `  y_p = np.clip(y_p, epsilon, 1 - epsilon)`
	 `  return -np.mean(y_a * np.log(y_p) + (1-y_a) * np.log(1-y_p))`

Categorical Cross-Entropy (CCE)	$$\mathcal{L}_{CCE} = -\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{C}(y_{ij} \log \hat{y}_{ij})$$
	 y_{ij} = Actual value of instance i for class j
	 $\hat{y}_{ij}$ = Predicted value of y_{ij}
	 C = Number of classes
	 n = Batch size
	 `def categorical_cross_entropy(self, y_a, y_p, epsilon=1e-15):`
	 `  y_p = np.clip(y_p, epsilon, 1-epsilon)`
	 `  return -np.mean(np.sum(y_a * np.log(y_p), axis = 1))`

Untuk neural network dengan L layer, gradient loss terhadap bobot di layer l dapat dihitung dengan chain rule:

Untuk Hidden Layer	**Untuk Output Layer**
	



Setiap layer akan menerima δ dari layer sesudahnya, menghitung kontribusinya, lalu meneruskan δ ke layer sebelumnya. Activation layer yang menerima gradient dari layer sesudahnya ($dout = \delta^{(l+1)}$) dan meneruskan gradient ke layer sebelumnya:

$$\delta^{(l)} = dout \odot activation'(z^{(l)})$$

Dimana $\odot$ adalah perkalian element-wise dan $activation'(z^{(l)})$ adalah turunan fungsi aktivasi yang dapat dilihat di tabel berikut:

Fungsi Aktivasi	**Formula dan Implementasi**
Linear	$$\frac{d(Linear(x))}{dx} = 1$$
	`def linear_derivative(self, x):` `  return 1`
Sigmoid	$$\frac{d(\sigma(x))}{dx} = \sigma(x)(1-\sigma(x))$$
	`def sigmoid_derivative(self, x):` `  return self.sigmoid(x) * (1 - self.sigmoid(x))`
ReLU	$$\frac{d(ReLU(x))}{dx} = \begin{cases} 0, & x \le 0 \\ 1, & x > 0 \end{cases}$$

	`def relu_derivative(self, x):`
	`  return np.where(x > 0, 1, 0)`

Hyperbolic Tangent	$$\frac{d(\tanh(x))}{dx} = \left(\frac{2}{e^{x} - e^{-x}}\right)^2$$ Note: setelah diperiksa, seharusnya operasi di penyebut adalah +, bukan -

	`def tanh_derivative(self, x):`
	`  return (2 /(np.exp(x) + np.exp(-x)))**2`

Softmax	Untuk vector $\dot{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$, $$\frac{d(softmax(\dot{x})_i)}{d\vec{x}} = \begin{bmatrix} \frac{\partial(softmax(\dot{x})_1)}{\partial x_1} & \cdots & \frac{\partial(softmax(\dot{x})_1)}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial(softmax(\vec{x})_n)}{\partial x_1} & \cdots & \frac{\partial(softmax(\vec{x})_n)}{\partial x_n} \end{bmatrix}$$ Dimana untuk $i, j \in \{1, \dots, n\}$, $$\frac{\partial(softmax(\dot{x})_i)}{\partial x_j} = softmax(\dot{x})_i(\delta_{i,j} - softmax(\dot{x})_j)$$ $$\delta_{i,j} = \begin{cases} 1, & i = j \\ 0, & i \neq j \end{cases}$$ Note: Softmax adalah kasus khusus, yaitu menggunakan Jacobian, bukan perkalian element-wise, sehingga perkalian gradient menggunakan matrix multiply, bukan element-wise.

	`def softmax_derivative(self, x):`
	`  S = self.softmax(x)`
	`  S_vector = S.reshape(-1,1)`
	`  diag = np.diagflat(S)`
	`  jacobian = diag - np.dot(S_vector, S_vector.T)`
	
	`  return jacobian`

Leaky ReLU	$$f'(x) = \begin{cases} 1 & x > 0, \\ \alpha & x \leq 0. \end{cases}$$

	`def leaky_relu_derivative(self, x, alpha=0.01):`
	`  return np.where(x>0, 1, alpha)`

Softplus	$$f'(x) = \frac{e^x}{1+e^x} = \sigma(x)$$ Note: turunan softplus adalah formula fungsi aktivasi sigmoid
	`def softplus_derivative(self, x):` `    return self.sigmoid(x)`

Setelah itu, kita masuk ke proses pembaruan bobot (weight update), dengan menggunakan gradient descent. Setiap backward pass, setiap Linear layer memiliki self.dw dan self.db. Optimizer menggunakan nilai ini untuk memperbarui bobot:

Gradient

$$\nabla E[\vec{w}] \equiv \left[\frac{\partial E}{\partial w_0}, \frac{\partial E}{\partial w_1}, \cdots \frac{\partial E}{\partial w_n}\right]$$

$$\vec{w} \leftarrow \vec{w} + \Delta\vec{w}$$

$$\Delta\vec{w} = -\eta \nabla E(\vec{w})$$

Implementasi Gradient Descent.step dapat dilihat sebagai berikut:

```
def step(self, layers: list) -> None:
    """
    Update weights and biases for all Linear layers in 'layers'
    """
    for layer in layers:
        if all(hasattr(layer, attr) for attr in ("w", "b", "dw", "db")):
            if layer.dw is None or layer.db is None:
                continue

            dw = layer.dw

            if self.reg_type == 'l1':
                dw = dw + self.lam * np.sign(layer.w)
            elif self.reg_type == 'l2':
                dw = dw + self.lam * layer.w

            layer.w -= self.lr * dw
            layer.b -= self.lr * layer.db
```

Namun, jika diperhatikan, di implementasi tersebut terdapat regularisasi. Regularisasi ini adalah metode yang digunakan sebagai

bentuk mitigasi overfitting di model machine learning. Regularisasi ini umumnya digunakan untuk data set yang memiliki fitur yang sangat banyak. Terdapat dua jenis Regularisasi:

Method	**Penjelasan**
L1 Regularization (Lasso)	$\sum_{i=1}^{n}(Y_i - \sum_{j=1}^{p} X_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} \lvert\beta_j\rvert$
	Lasso (Least Absolute Shrinkage and Selection Operator) akan mendorong bobot menjadi tepat nol (sparse), dengan penalty term berupa *absolute value of the sum of coefficients.*
L2 Regularization (Ridge)	$\sum_{i=1}^{n}(y_i - \sum_{j=1}^{p} x_{ij}\beta_j)^2 + \lambda \sum_{j=1}^{p} \beta_j^2$
	Ridge akan mendorong bobot menjadi kecil tanpa benar-benar nol dengan penalty term berupa *squared sum of coefficients*.

Implementasi FFNN.backward dapat dilihat sebagai berikut:

```
def backward(self, y_pred: np.ndarray, y_true: np.ndarray) -> float:
    loss = self.loss_layer.forward(y_pred, y_true)
    grad = self.loss_layer.backward(y_pred, y_true)

    for layer in reversed(self.layers):
        grad = layer.backward(grad)

    return loss
```

Terdapat tiga langkah utama:

1. Hitung loss di forward
2. Hitung gradient awal dari loss
3. Propagasikan gradient dari output ke input.

Dan berikut adalah siklus training lengkap (train_step):

```
def train_step(self, x_batch: np.ndarray, y_batch: np.ndarray, optimizer) -> float:
    y_batch = np.asarray(y_batch, dtype=np.float64).reshape(-1, 1)
    y_pred = self.forward(x_batch)
    loss = self.backward(y_pred, y_batch)
    optimizer.step(self.layers)
    return loss
```

2.2 Hasil Pengujian

Untuk menguji FFNN yang sudah diimplementasikan, telah dilakukan training beberapa model dengan *hyperparameter* yang berbeda-beda. Data yang digunakan untuk pengujian ini adalah datasetml_2026.csv dengan **50 *epoch*** dan ukuran ***batch* 32**. Berikut adalah hasil pengujian kami:

2.2.1 Pengaruh Depth dan Width

Notebook Pengujian: depth_width_analysis.ipynb

Tabel 2.2.1.1 Tabel **Analisis *Width***

No	Hasil Pengujian

1	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[32, 32, 32],`
	`output_dim=1,`
	`hidden_activation='relu',`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Width-32: accuracy = 0.7345

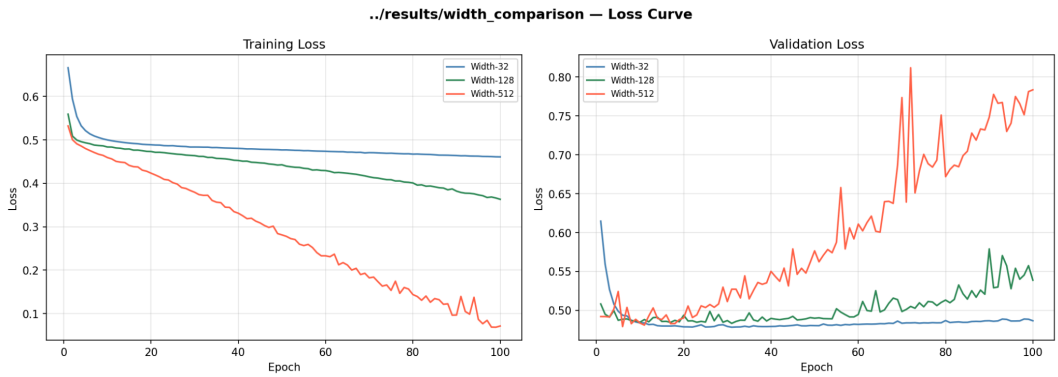
2	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[128, 128, 128],`
	`output_dim=1,`
	`hidden_activation='relu',`
	`output_activation='sigmoid',`
	`loss_name='bce',`

	`init_method='he',` `seed=42`
	Hasil: Width-128: accuracy = 0.7165

3	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[512, 512, 512],`
	`output_dim=1,`
	`hidden_activation='relu',`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Width-512: accuracy = 0.7015

Penjelasan:



Gambar 2.2.1.1 Kurva loss (width)

Dari grafik *training loss*, terlihat pola yang sangat jelas bahwa semakin lebar jaringan semakin cepat dan dalam *training loss*-nya turun. Width-512 berhasil menekan *training loss* hingga mendekati 0.07 di epoch 100, jauh lebih rendah dibanding Width-128 yang berhenti di sekitar 0.36 dan Width-32 yang stagnan di 0.46. Ini menunjukkan bahwa model yang lebih lebar memiliki **kapasitas yang jauh lebih besar** untuk mempelajari pola dari data training.

Namun validation loss menceritakan kisah yang sangat berbeda. Width-32 justru memiliki validation loss paling stabil dan rendah sepanjang 100 epoch, bertahan di sekitar 0.49 tanpa fluktuasi berarti. Width-128 relatif stabil hingga epoch 70 namun mulai sedikit naik di akhir. Width-512 adalah yang paling bermasalah *validation loss*-nya mulai merangkak naik sejak epoch 20, kemudian meledak tidak stabil dengan lonjakan-lonjakan besar hingga mencapai 0.80 di epoch 100. Ini adalah tanda ***overfitting* yang sangat parah** pada Width-512 model terlalu besar untuk ukuran dataset ini sehingga ia menghafal data *training* alih-alih belajar pola yang dapat digeneralisasi.

Pola ini dikonfirmasi oleh hasil akurasi pada data test. Width-32 mendapat akurasi tertinggi (0.7345), diikuti Width-128 (0.7165), dan Width-512 justru yang terburuk (0.7015) meskipun *training loss*-nya paling rendah. Fenomena ini dikenal sebagai ***generalization gap*** antara performa di *training set* dan *test set* semakin besar seiring bertambahnya kapasitas model. Width-512 dengan 512 neuron per layer memiliki ratusan ribu parameter yang jauh melebihi kebutuhan untuk dataset ini, sehingga model belajar *noise* dan detail spesifik dari training data yang tidak berlaku pada data baru.

Kesimpulannya, untuk dataset ini model yang lebih kecil dan sederhana justru lebih baik dalam generalisasi. Width-32 dengan kapasitas terbatas terbukti paling optimal karena kompleksitas dataset tidak memerlukan jaringan yang sangat lebar. Jika ingin meningkatkan performa Width-512, solusinya adalah menambahkan regularisasi L1 atau L2 untuk mengontrol *overfitting*, atau mengurangi jumlah epoch dengan early stopping.

Tabel 2.2.1.2 Tabel **Analisis *Depth***

No	Hasil Pengujian

1	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[128, 128],`
	`output_dim=1,`
	`hidden_activation='relu',`

	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Depth-2: accuracy = 0.7245

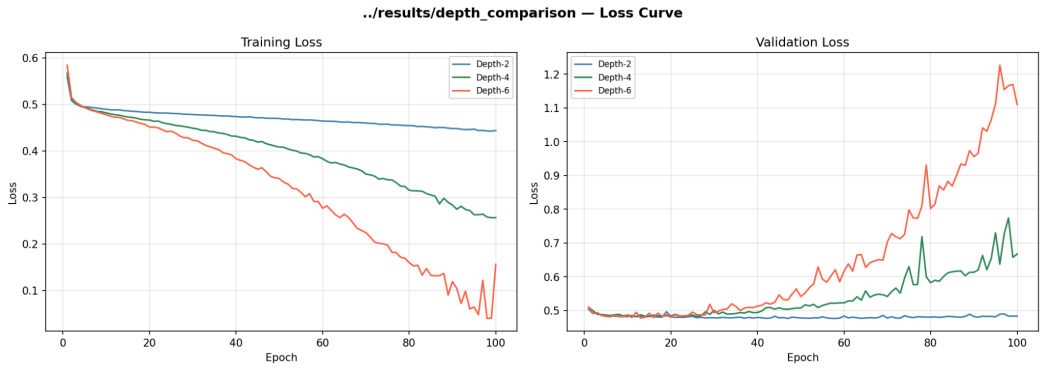
2	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[128, 128, 128, 128],`
	`output_dim=1,`
	`hidden_activation='relu',`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Depth-4: accuracy = 0.6980

3	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[128, 128, 128, 128, 128, 128],`
	`output_dim=1,`
	`hidden_activation='relu',`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Depth-6: accuracy = 0.6885

Penjelasan:



Gambar 2.2.1.2 Kurva loss (depth)

Dari grafik *training loss*, polanya sangat mirip dengan yang terjadi pada *width comparison* semakin dalam jaringan semakin agresif *training loss*-nya turun. Depth-6 berhasil menekan *training loss* hingga mendekati 0.04 di epoch 100 dengan penurunan yang konsisten dan curam, Depth-4 turun lebih moderat hingga sekitar 0.26, sementara Depth-2 hampir stagnan di 0.45 sejak epoch 20. Kapasitas representasi yang lebih besar pada jaringan yang lebih dalam memungkinkan model untuk mempelajari pola yang jauh lebih kompleks dari data training.

Namun seperti pada kasus *width*, validation loss mengungkap masalah yang serius pada model yang lebih dalam. Depth-2 kembali menjadi yang paling stabil *validation loss*-nya datar di sekitar 0.49 sepanjang 100 epoch tanpa fluktuasi berarti. Depth-4 mulai menunjukkan kenaikan *validation loss* sejak epoch 40 dan terus naik hingga 0.65 di epoch 100 disertai lonjakan tajam di sekitar epoch 80. Depth-6 adalah yang paling ekstrem validation loss-nya mulai merangkak naik sejak epoch 30 dan meledak hingga melampaui 1.2 di epoch 95-100, dengan fluktuasi yang sangat tidak stabil. Skala validation loss Depth-6 bahkan jauh melebihi kasus Width-512 sebelumnya, menunjukkan bahwa ***overfitting*** akibat penambahan depth lebih parah dibanding akibat penambahan width pada dataset ini.

Hasil akurasi pada data test sepenuhnya konsisten dengan observasi tersebut. Depth-2 mendapat akurasi tertinggi (0.7245), Depth-4 turun cukup signifikan (0.6980), dan Depth-6 mendapat akurasi terendah dari semua model yang telah diuji sejauh ini (0.6885). Yang

menarik, penurunan akurasi dari Depth-2 ke Depth-6 mencapai sekitar **3.6%** lebih besar dibanding penurunan dari Width-32 ke Width-512 yang hanya sekitar **3.3%**. Ini memperkuat kesimpulan bahwa penambahan depth memberikan dampak ***overfitting*** **yang lebih destruktif dibanding penambahan width untuk dataset ini**.

Fenomena ini dapat dijelaskan dari perspektif vanishing gradient semakin dalam jaringan, gradien yang mengalir balik ke *layer-layer* awal semakin kecil karena terdilusi oleh banyaknya perkalian matriks selama backward pass. Akibatnya *layer-layer* awal hampir tidak belajar sama sekali sementara *layer-layer* akhir *overfitting* pada data training. Hal ini diperparah oleh fakta bahwa dataset ini relatif kecil dan sederhana sehingga **arsitektur 6 hidden layer dengan 128 neuron per layer memiliki kapasitas yang sangat berlebihan**.

Secara keseluruhan, baik dari pengujian width maupun depth, terbukti bahwa arsitektur yang lebih sederhana dan dangkal justru lebih optimal untuk dataset ini. Model dengan 2-3 hidden layer dan width sedang (32-128) memberikan keseimbangan terbaik antara kapasitas belajar dan kemampuan generalisasi. Untuk menggunakan model yang lebih dalam atau lebih lebar secara efektif, diperlukan teknik seperti regularisasi L1/L2, *dropout*, atau *early stopping* untuk mengendalikan *overfitting*.

2.2.2 Pengaruh Fungsi Aktivasi

Notebook Pengujian: activation_analysis.ipynb

No	Hasil Pengujian

1	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[256, 128, 64],`
	`output_dim=1,`
	`hidden_activation='linear',`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Linear: accuracy = 0.7395

2	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[256, 128, 64],`
	`output_dim=1,`
	`hidden_activation=’relu’,`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: ReLU: accuracy = 0.7240

3	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[256, 128, 64],`
	`output_dim=1,`
	`hidden_activation=’sigmoid’,`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: Sigmoid: accuracy = 0.7375

4	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[256, 128, 64],`
	`output_dim=1,`
	`hidden_activation=’tanh’,`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

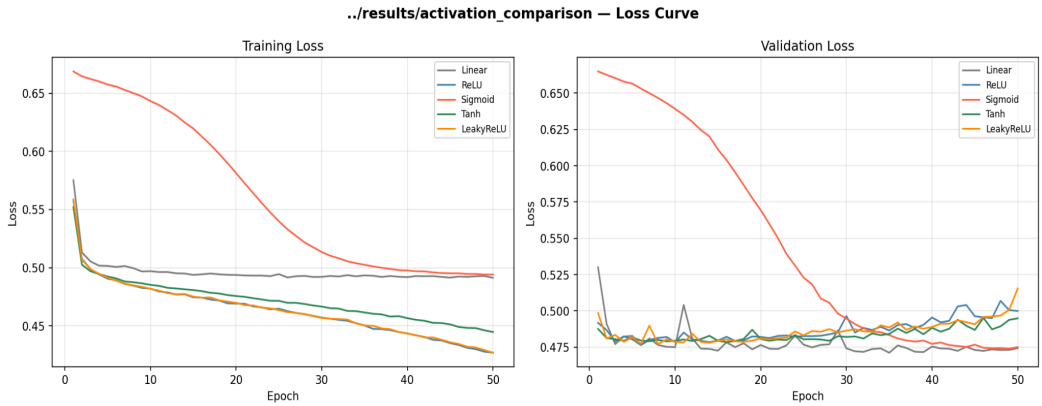
	Hasil: Tanh: accuracy = 0.7340

5	Hyperparameter:
	`input_dim=INPUT_DIM,`
	`hidden_dim=[256, 128, 64],`
	`output_dim=1,`

	`hidden_activation=’leaky_relu’,`
	`output_activation='sigmoid',`
	`loss_name='bce',`
	`init_method='he',`
	`seed=42`

	Hasil: LeakyReLU: accuracy = 0.7275

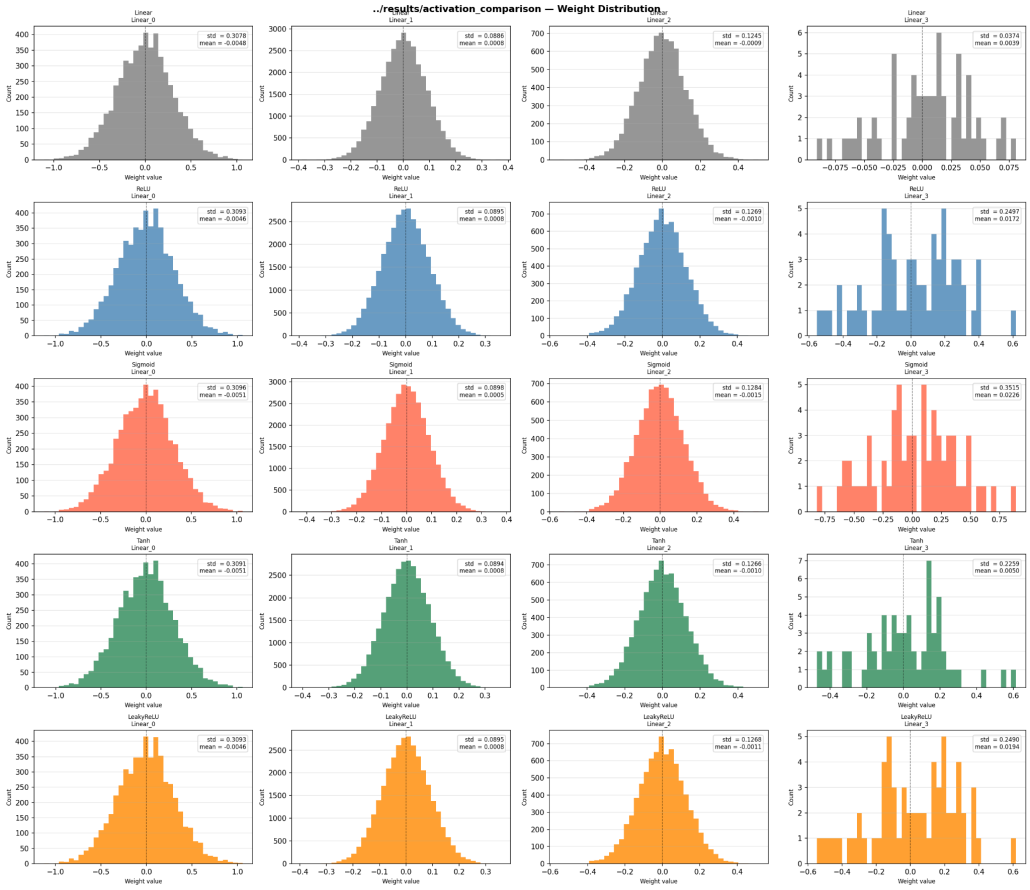
Penjelasan:



Gambar 2.2.2.1 Kurva Loss

Dari grafik training loss, terlihat jelas bahwa Sigmoid memulai dengan loss tertinggi (~0.67) dan konvergen paling lambat, baru mulai turun signifikan di sekitar epoch 15-20 dan bahkan di epoch 50 masih belum sepenuhnya konvergen seperti model lainnya. Ini adalah gejala klasik yakni ***vanishing gradient*** pada Sigmoid yang disebabkan karena gradiennya kecil sehingga update bobot di layer-layer awal sangat lambat. Sebaliknya, Linear, ReLU, Tanh, dan LeakyReLU sudah konvergen cepat di epoch 1-5 pertama.

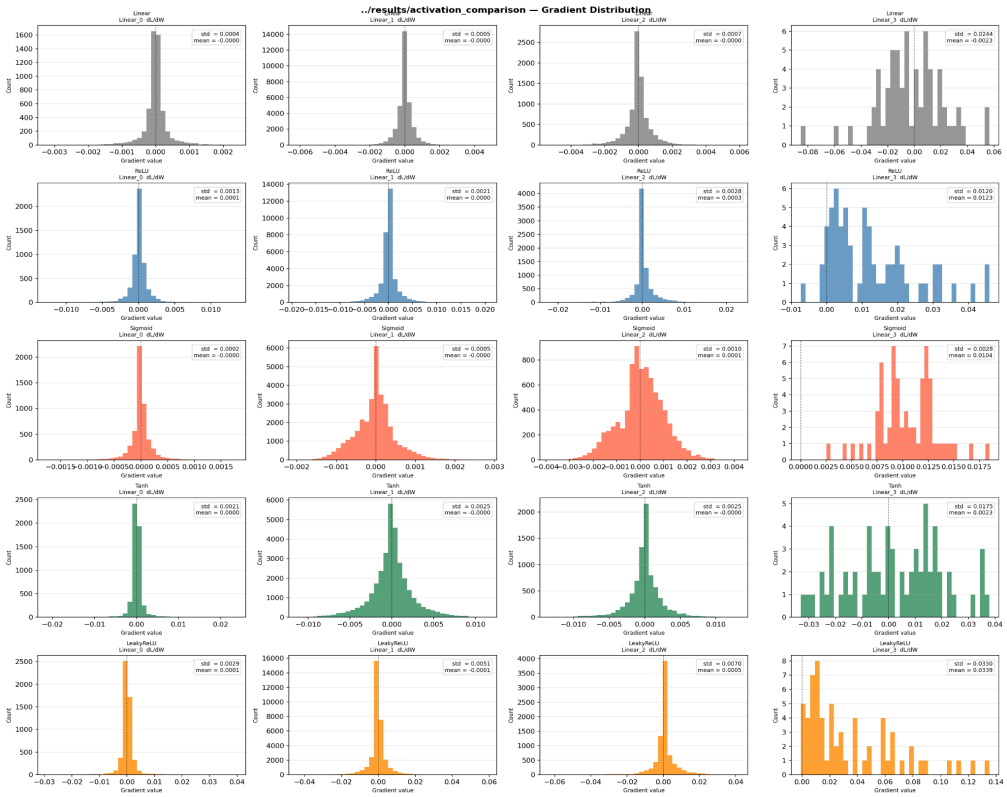
Pada validation loss, semua model berkumpul di rentang 0.475 - 0.500 setelah epoch 30, yang menunjukkan bahwa semua model sebenarnya mencapai kemampuan generalisasi yang serupa. Validation loss yang sedikit naik-turun (karena *noise*) di semua model wajar karena ukuran validation set yang relatif kecil.



Gambar 2.2.2.2 Grafik distribusi *weight* (bobot)

Distribusi bobot di Linear_0, Linear_1, Linear_2 (hidden layers) terlihat hampir identik di semua model dengan std sekitar 0.30, 0.09, dan 0.12. Ini masuk akal karena semua model menggunakan init_method='he' dan seed=42 yang sama, sehingga bobot awalnya identik. Perbedaan antara model di layer-layer ini sangat kecil setelah training, menunjukkan bahwa variasi fungsi aktivasi tidak terlalu mengubah distribusi bobot di hidden layer secara drastis.

Yang menarik adalah di Linear_3 (output layer) yakni di sini distribusi mulai berbeda antar model. Sigmoid memiliki std 0.3515 yang paling besar dengan sebaran lebar ke kiri (-0.75 hingga 0.75), sementara Linear memiliki std paling kecil (0.0374) dan terpusat sempit. Ini mencerminkan bahwa Sigmoid membutuhkan bobot output yang lebih besar untuk mengkompensasi sinyal yang lebih kecil dari hidden layer akibat saturasi fungsi aktivasi.



Gambar 2.2.2.3 Grafik distribusi gradien

Berdasarkan visualisasi di atas, Linear_0, Linear_1, Linear_3 (kolom kiri pojok hingga ketiga), semua model menunjukkan distribusi gradien yang sangat sempit dan terpusat di sekitar nol dengan std yang sangat kecil (0.0002 hingga 0.007). Namun, Sigmoid secara konsisten memiliki std paling kecil di antara semua model misalnya di Linear_0 std Sigmoid hanya 0.0002 dibanding ReLU 0.0013 dan LeakyReLU 0.0029. Ini adalah bukti kuat *vanishing gradient* pada Sigmoid, yakni gradien mengecil drastis saat melewati banyak layer ke belakang.

Di Linear_3 (output layer, kolom paling kanan), polanya berbalik yakni semua model memiliki gradien yang lebih tersebar dan LeakyReLU memiliki std tertinggi (0.0330) diikuti ReLU (0.0120), sedangkan Sigmoid justru memiliki std rendah (0.0028). Ini konsisten dengan fakta bahwa LeakyReLU dan ReLU meneruskan gradien lebih besar ke output sementara Sigmoid meredam gradien.

Jadi, dari hasil akurasi model pun menarik karena **Linear** justru mendapatkan akurasi tertinggi meski fungsi aktivasinya paling sederhana, ini memungkinkan karena datasetnya cukup ***linearly separable*** sehingga

non-linearity dari fungsi aktivasi yang kompleks tidak memberikan keuntungan yang signifikan.

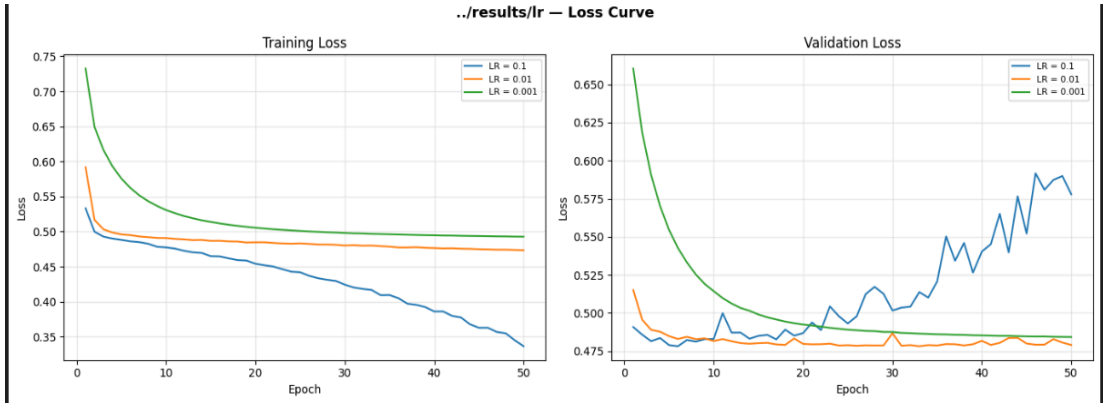
2.2.3 Pengaruh Learning Rate

Notebook Pengujian: learning_rate_analysis.ipynb

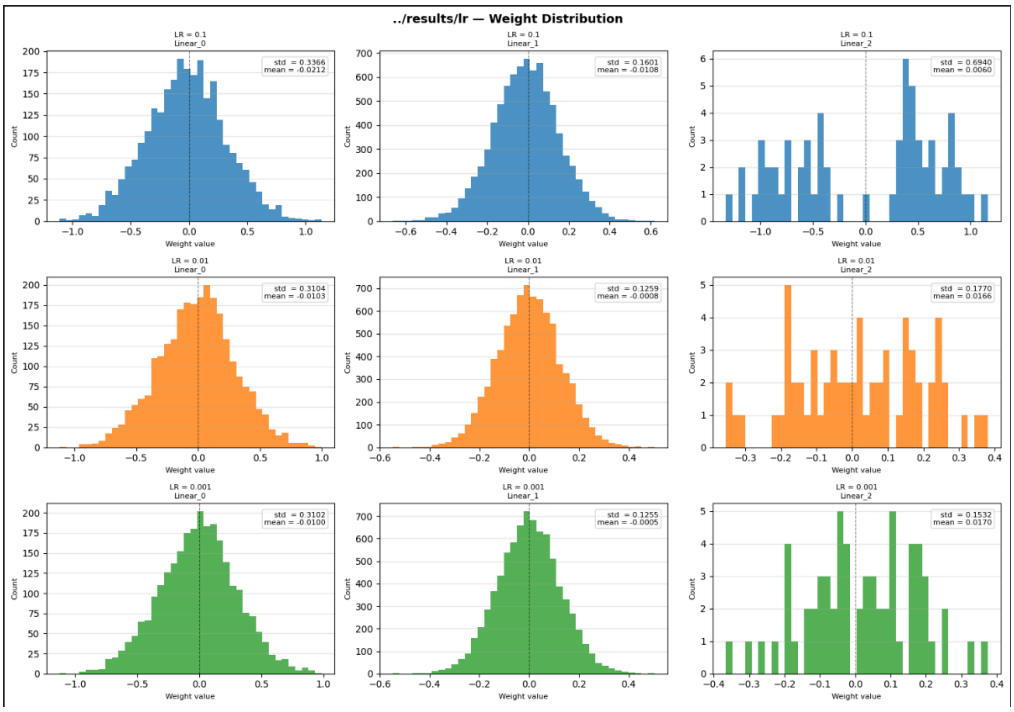
No	Hasil Pengujian
1	Hyperparameter: hidden_dim = [128, 64], output_dim = 1, hidden_activation = HIDDEN_ACTIVATION, output_activation = OUTPUT_ACTIVATION, loss_name = LOSS_NAME, init_method = 'zero', seed = SEED, Lr =0.1
	Hasil: Test Accuracy: 0.6960
2	Hyperparameter: Hyperparameter: hidden_dim = [128, 64], output_dim = 1, hidden_activation = HIDDEN_ACTIVATION, output_activation = OUTPUT_ACTIVATION, loss_name = LOSS_NAME, init_method = 'zero', seed = SEED, Lr =0.01
	Hasil: Test Accuracy: 0.7445
3	Hyperparameter: Hyperparameter: hidden_dim = [128, 64],

	output_dim = 1, hidden_activation = HIDDEN_ACTIVATION, output_activation = OUTPUT_ACTIVATION, loss_name = LOSS_NAME, init_method = 'zero', seed = SEED, Lr =0.001
	Hasil: Test Accuracy: 0.7400

Penjelasan:

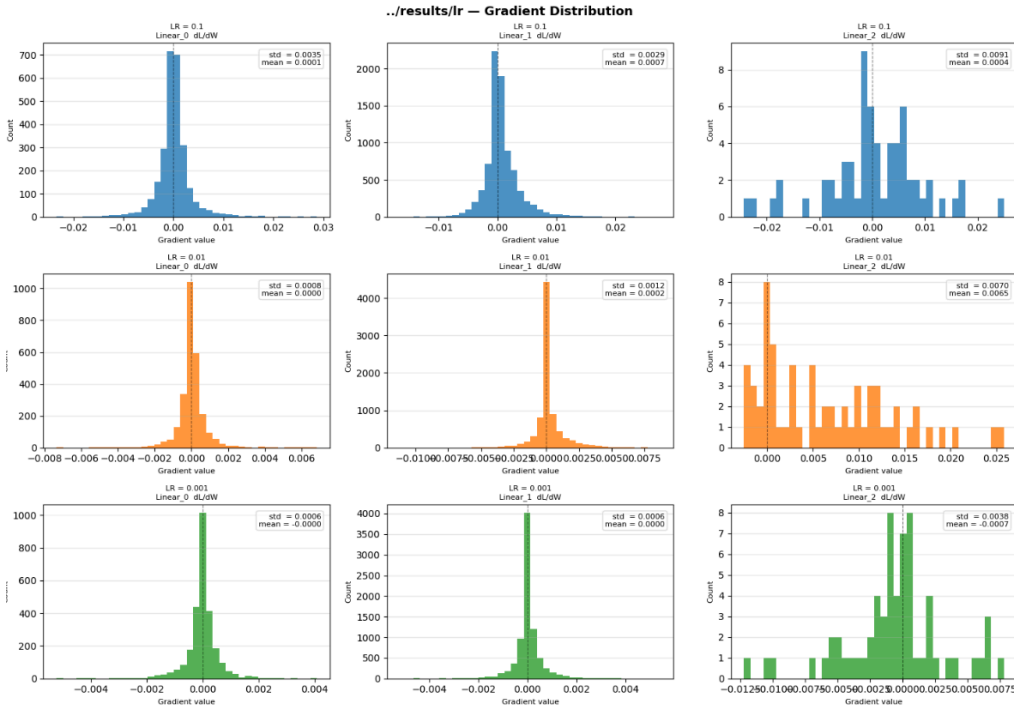


Pada eksperimen ini dapat dilihat bahwa pada Learning Rate 0.1 terjadi overfitting, dapat dilihat pada grafik bahwa training loss terus menurun, sedangkan validation loss turun sesaat sebelum akhirnya melonjak tajam setelah epoch 10. Hal ini karena model terlalu mengikuti data training sehingga kemampuan menurun dan accuracy rendah. Eksperimen dengan Learning Rate 0.01 mendapat hasil yang stabil, training loss dan validation loss turun dan konvergen di sekitar 0.47-0.48 tanpa divergensi. Accuracy yang dihasilkan cukup tinggi sebesar 74.45%. Sedangkan untuk learning rate 0.001 memulai dengan loss yang tinggi ~0.73 dan loss yang rendah.



Pada hasil eksperimen dapat dilihat pada Plot weight distribution Linear_0 dan Linear_1 distribusi bobot ketiga model cukup mirip dengan membentuk kurva simetris dengan pusat di sekitar nol. Dapat dilihat learning rate 0.1 memiliki std sedikit lebih besar (linear_0 = 0.3366, Linear_1 = 0.1601) dibandingkan dengan Learning rate 0.01 dan 0.001.

Perbedaan terlihat pada Linear_2, pada learning rate 0.1 terjadi deviasi standar yang membesar hingga 0.6940 sengat lebar jika dibandingkan dengan learning rate 0.01 sekitar 0.1770 dan Learning rate 0.001 sekitar 0.1532. Hal ini mengkonfirmasi bahwa learning rate yang besar mendorong bobot bergerak secara agresif.



Distribusi gradien yang dihasilkan berbanding lurus dengan besaran learning rate yang digunakan. Pada *learning rate* 0.1, varians gradien sangat lebar yang menandakan ketidakstabilan langkah optimasi selama pelatihan. Sebaliknya, *learning rate* 0.01 dan 0.001, distribusi gradien jauh lebih terpusat di angka nol dengan deviasi standar yang sangat kecil. Kondisi ini membuktikan terjadinya proses backpropagation yang terkontrol dan stabil, yang memungkinkan model melakukan penyesuaian bobot secara presisi tanpa merusak representasi fitur yang sudah dipelajari pada epoch-epoch awal.

2.2.4 Pengaruh Inisialisasi Bobot

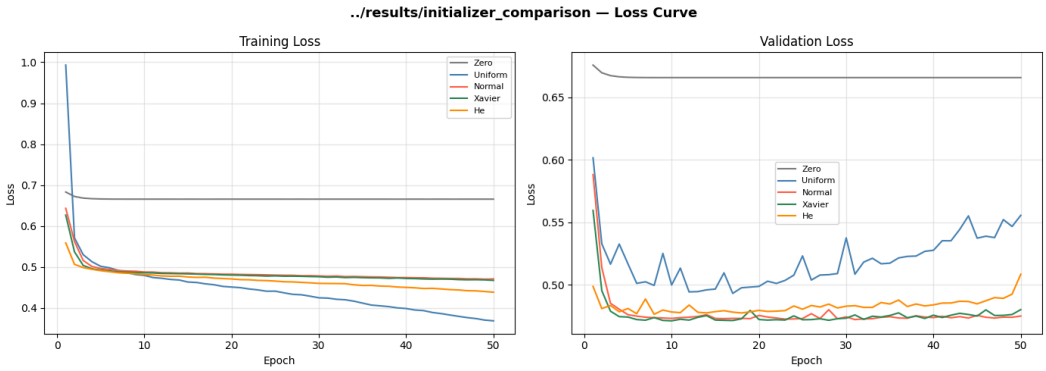
Notebook Pengujian: weight_initializer_analysis.ipynb

No	Hasil Pengujian
1	Hyperparameter: HIDDEN_DIM = [128, 64] EPOCHS = 500 BATCH_SIZE = 64 LR = 0.01

	HIDDEN_ACTIVATION = 'relu' OUTPUT_ACTIVATION = 'sigmoid' LOSS_NAME = 'bce' SEED = 42 INIT_METHOD = ‘zero’
	Hasil: Test Accuracy: 0.6110
2	Hyperparameter: HIDDEN_DIM = [128, 64] EPOCHS = 500 BATCH_SIZE = 64 LR = 0.01 HIDDEN_ACTIVATION = 'relu' OUTPUT_ACTIVATION = 'sigmoid' LOSS_NAME = 'bce' SEED = 42 INIT_METHOD = ‘uniform’
	Hasil: Test Accuracy: 0.7150
3	Hyperparameter: HIDDEN_DIM = [128, 64] EPOCHS = 500 BATCH_SIZE = 64 LR = 0.01 HIDDEN_ACTIVATION = 'relu' OUTPUT_ACTIVATION = 'sigmoid' LOSS_NAME = 'bce' SEED = 42 INIT_METHOD = ‘normal’
	Hasil: Test Accuracy: 0.7020
4	Hyperparameter: HIDDEN_DIM = [128, 64] EPOCHS = 500 BATCH_SIZE = 64 LR = 0.01

	HIDDEN_ACTIVATION = 'relu' OUTPUT_ACTIVATION = 'sigmoid' LOSS_NAME = 'bce' SEED = 42 INIT_METHOD = ‘xavier’
	Hasil: Test Accuracy: 0.7365
5	Hyperparameter: HIDDEN_DIM = [128, 64] EPOCHS = 500 BATCH_SIZE = 64 LR = 0.01 HIDDEN_ACTIVATION = 'relu' OUTPUT_ACTIVATION = 'sigmoid' LOSS_NAME = 'bce' SEED = 42 INIT_METHOD = ‘he’
	Hasil: Test Accuracy: 0.7310

Penjelasan:



Gambar 2.2.4.1 Kurva Loss untuk semua Weight Initializer

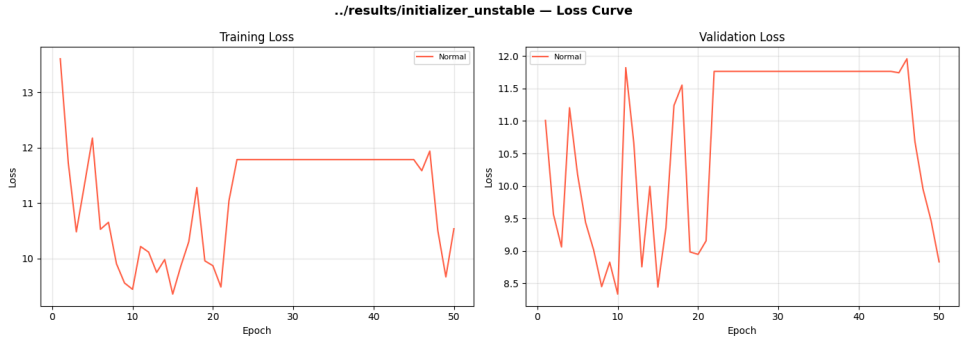
Dari kurva loss untuk zero sebagai weight init dapat dilihat kalau loss turun sangat lambat dan val acc tidak pernah bergerak dari 0.6162 selama 50 epoch.

Untuk Random with Uniform Distribution, epoch pertama memiliki train loss yang sangat tinggi, kemudian turun agresif. Pada epoch 50, train

loss mencapai 0.3688 (paling rendah), namun val loss sudah naik dari epoch ~15.

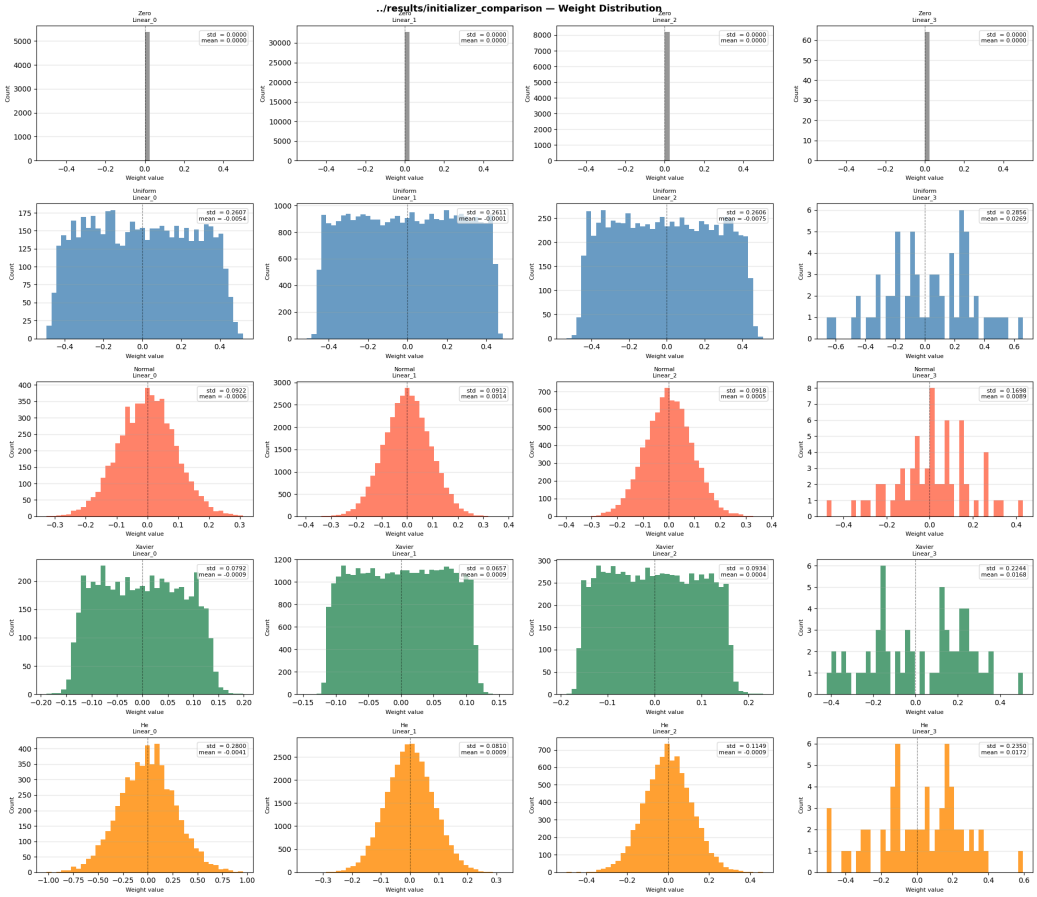
Untuk Random with Normal Uniform, konvergen sangat smooth, dimana epoch 1 menghasilkan val loss 0.5882, dan turun stabil ke 0.4750 di epoch 50. Tidak ada lonjakan/osilasi seperti weight init lainnya dan val loss di epoch 50 adalah yang terendah di antara semua weight init.

Untuk Xavier, konvergen mulus dan konsisten, dengan loss epoch 50 0.4802, sedikit di atas Normal. Dan terakhir untuk He, memiliki konvergen cepat di awal, tapi mulai naik perlahan juga dari epoch ~20.



Gambar 2.2.4.2 Kurva Loss untuk Random with Normal Distribution with High Variance

Terdapat penemuan menarik selama proses pengujian, yaitu saat menguji dengan variance = 1.0, terdapat osilasi yang tidak stabil untuk training loss dan validation loss untuk weight init random with normal distribution, namun untuk weight ini lain masih mirip dengan ketika diuji dengan variance = 0.001. Ini menunjukkan kalau variance menjadi parameter yang paling kritis pada weight init ini.

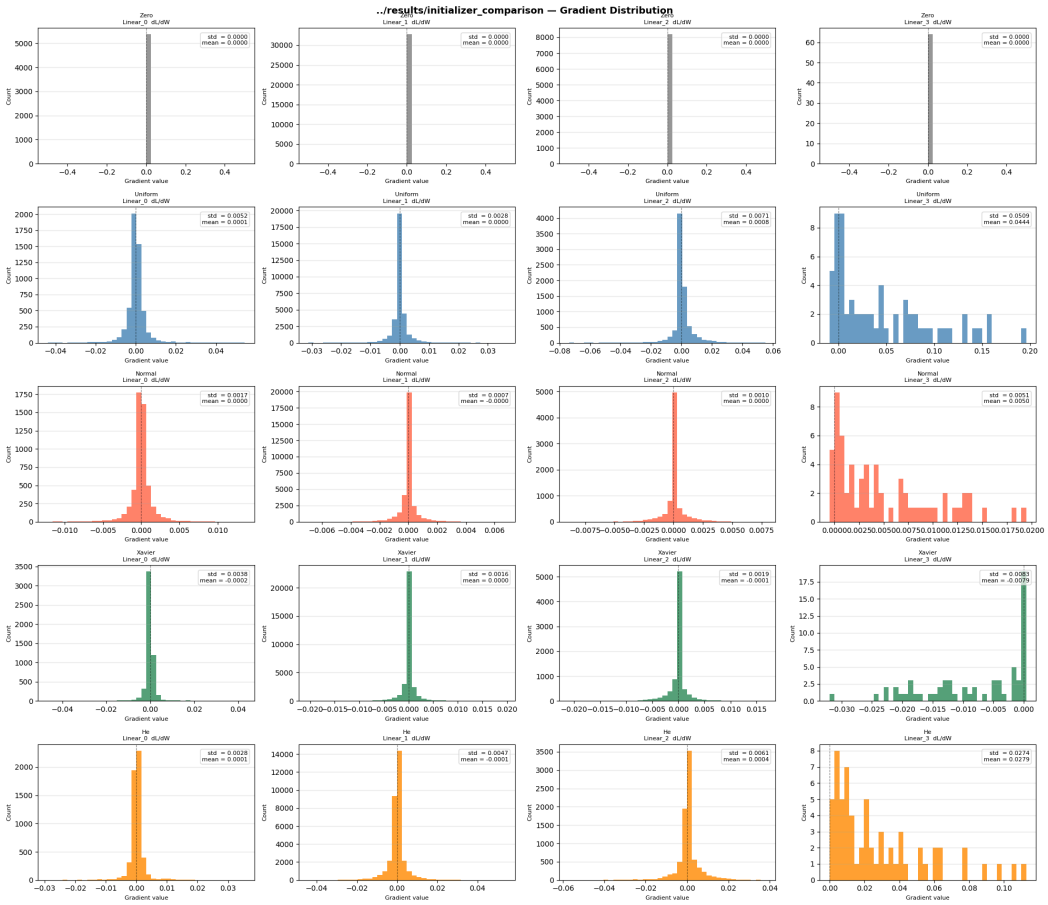


Gambar 2.2.4.3 Weight Distribution untuk semua Weight Initializer

Untuk zero, bobot sangat terkonsentrasi di sekitar nol dengan spread sangat sempit. Meski loss sedikit turun, gradien yang identik antar neuron membuat semua bobot bergerak ke arah yang sama sehingga distribusi tetap sempit dan tidak informatif.

Untuk uniform, distribusi cukup lebar dan karena memiliki initial range [-0.5, 0.5] dan ini cukup besar sehingga bobot memiliki ruang untuk berdiferensiasi dari awal dan setelah training, distribusi condong ke nilai positif kecil. Untuk normal, distribusi awal sempit, namun masih cukup untuk memberikan diferensiasi antar neuron. Distribusinya juga masih melebar secara teratur dan simetris yang menunjukkan bobot berkembang secara sehat.

Untuk Xavier, sedikit mirip dengan uniform, tetapi setelah training, sedikit condong ke nilai negatif. Dan untuk He, mirip dengan normal, tetapi setelah training, distribusinya melebar dan tidak berada di titik 0 seperti normal.



Gambar 2.2.4.4 Gradient Distribution untuk semua Weight Initializer

Untuk zero, gradien hampir nol di semua layer, mirip dengan weight distribution. Untuk uniform, dari layer 0, 1, 2, gradien mengalir sangat kecil (dapat dilihat dari std-nya) dan memiliki nilai positif, tetapi loncat besar di layer 3. Begitu juga di He. Dan untuk normal juga loncat di Layer 3, tetapi tidak sebesar uniform dan He. Tetapi sangat berbeda dengan xavier, dimana ini satu-satunya initializer yang outputnya malah negatif.

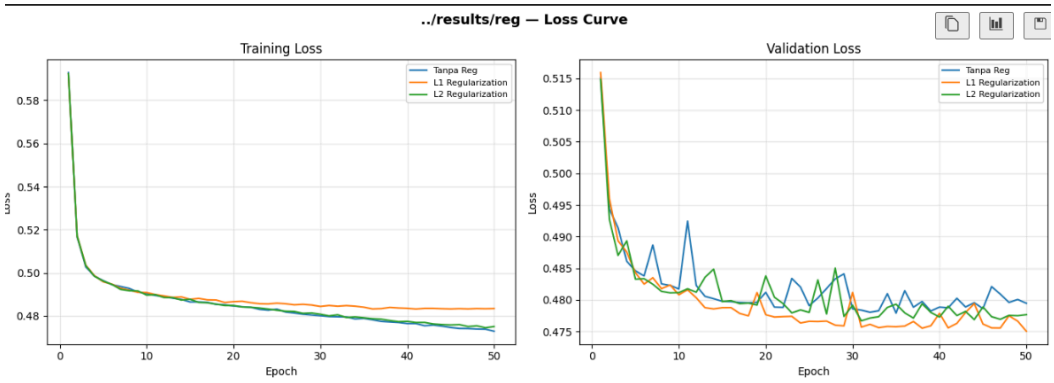
2.2.5 Pengaruh Regularisasi

Notebook Pengujian: regularization_analysis.ipynb

No	Hasil Pengujian

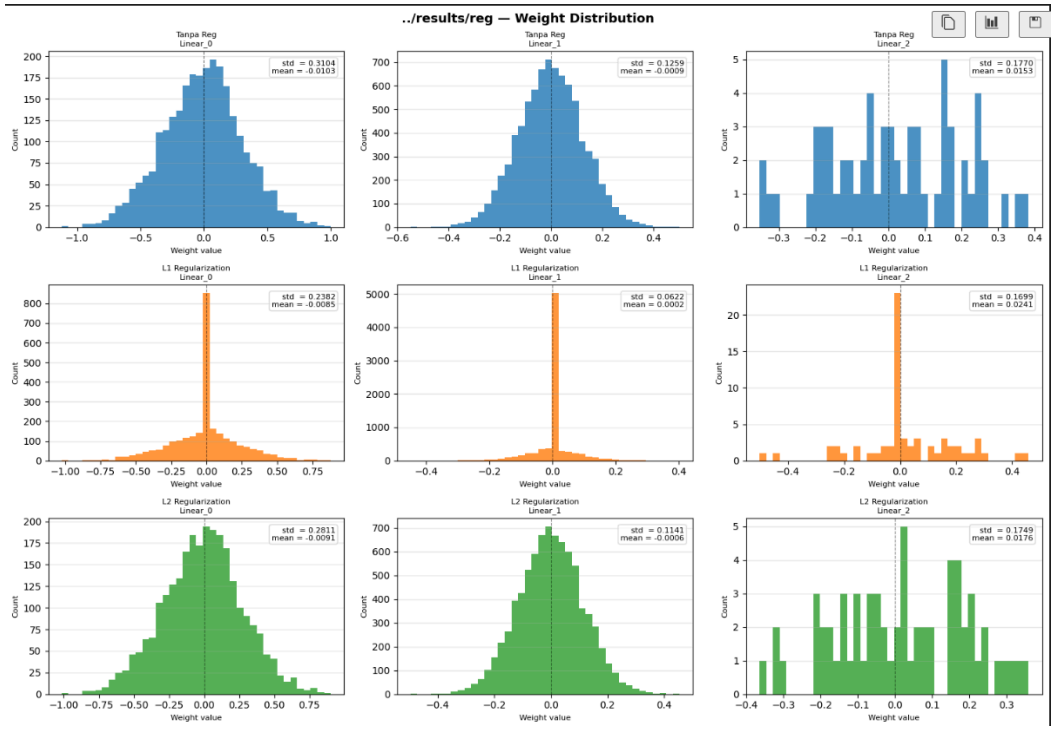
1	Hyperparameter: hidden_dim = [128, 64], output_dim = 1, hidden_activation = HIDDEN_ACTIVATION, output_activation = OUTPUT_ACTIVATION, loss_name = LOSS_NAME, init_method = 'zero', seed = SEED, Reg_type =None
	Hasil: Test Accuracy: 0.7420
2	Hyperparameter: hidden_dim = [128, 64], output_dim = 1, hidden_activation = HIDDEN_ACTIVATION, output_activation = OUTPUT_ACTIVATION, loss_name = LOSS_NAME, init_method = 'zero', seed = SEED, Reg_type =l1 0.001
	Hasil: Test Accuracy: 0.7390
3	Hyperparameter: hidden_dim = [128, 64], output_dim = 1, hidden_activation = HIDDEN_ACTIVATION, output_activation = OUTPUT_ACTIVATION, loss_name = LOSS_NAME, init_method = 'zero', seed = SEED, Reg_type = l2 0.001
	Hasil: Test Accuracy: 0.7380

Penjelasan:

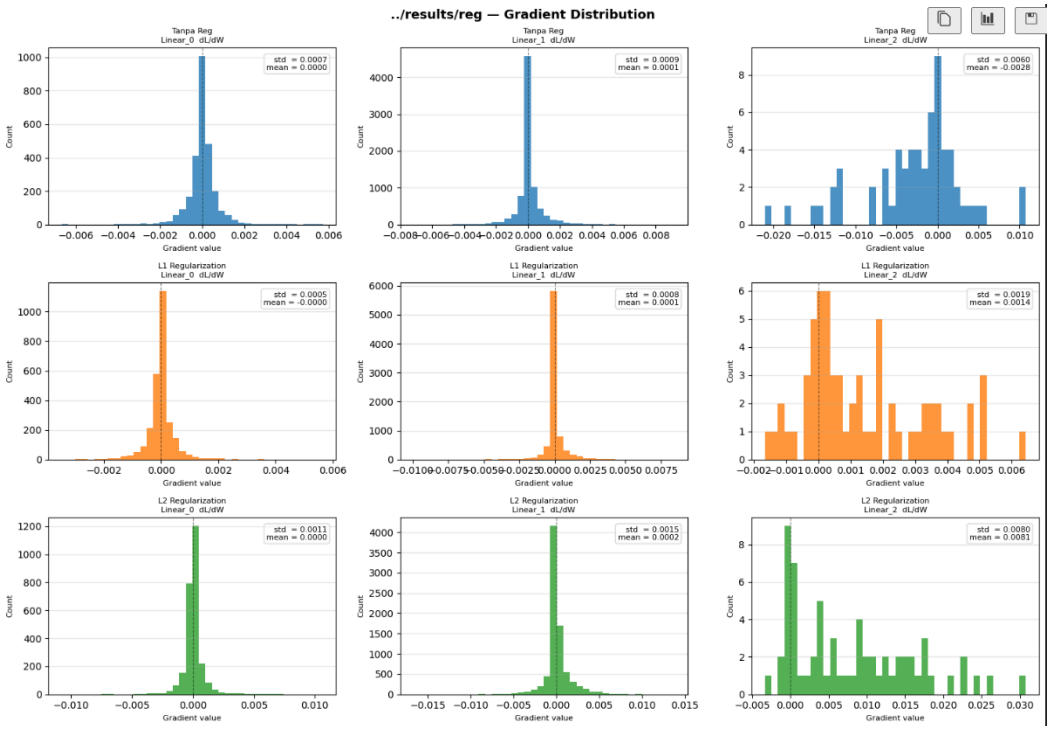


Pada eksperimen ini dapat dilihat bahwa grafik training loss untuk ketiga model hampir identik, ketiganya turun cepat di epoch 1-10 kemudian melambat di sekitar 0.47-0.48. Dapat dilihat bahwa L1 memiliki training loss yang sedikit lebih tinggi dibandingkan dengan Tanpa regularisasi maupun L2 regularisasi.

Pada grafik validation loss, ketiga model memiliki rentang yang berdekatan di sekitar 0.475-0.480. L1 menunjukkan validation loss paling rendah (~0.475) dibandingkan tanpa regularisasi dan L2. Tidak terlihat tanda overfitting yang signifikan pada ketiga model karena selisih train loss dan val loss sangat kecil, mengindikasikan bahwa dengan arsitektur [128, 64] dan 50 epoch, model belum mengalami overfitting sehingga manfaat regularisasi belum terlihat signifikan.



Model tanpa regularisasi memiliki distribusi normal, menentukan bobot tersebar dengan bebas mengikuti sinyal gradient tanpa batas.Model L1 regularisasi menghasilkan distribusi yang lancip sangat jauh jika dibandingkan dengan distribusi model lainnya. Ini adalah tanda L1 yang mendorong banyak bobot menuju tepat nol (*sparsity*), sehingga banyak neuron di hidden layer menjadi tidak aktif. Sedangkan, L2 menghasilkan distribusi yang lebih terpusat dibanding model lainnya.



Distribusi gradien pada Linear_0 dan Linear_1 ketiga model sangat mirip dan terpusat di sekitar nol. Tanpa Reg memiliki std gradien terbesar (Linear_0 std=0.0007, Linear_1 std=0.0009), diikuti L2 (std=0.0011, 0.0015) yang sedikit lebih besar karena pengaruh penalty L2 pada gradien, kemudian L1 (std=0.0005, 0.0008) yang paling kecil konsisten dengan bobot yang lebih sparse.

Perbedaan paling terlihat pada Linear_2 (output layer) — Tanpa Reg memiliki gradien yang tersebar ke kiri (mean=-0.0028, std=0.0060), menunjukkan bobot output masih aktif dikoreksi. L1 memiliki gradien yang sudah sangat kecil dan tersebar merata (std=0.0019), sementara L2 memiliki gradien yang justru tersebar ke kanan dengan mean=0.0081 dan std=0.0080, menunjukkan bobot output L2 masih terus dioptimalkan ke arah positif di akhir training.

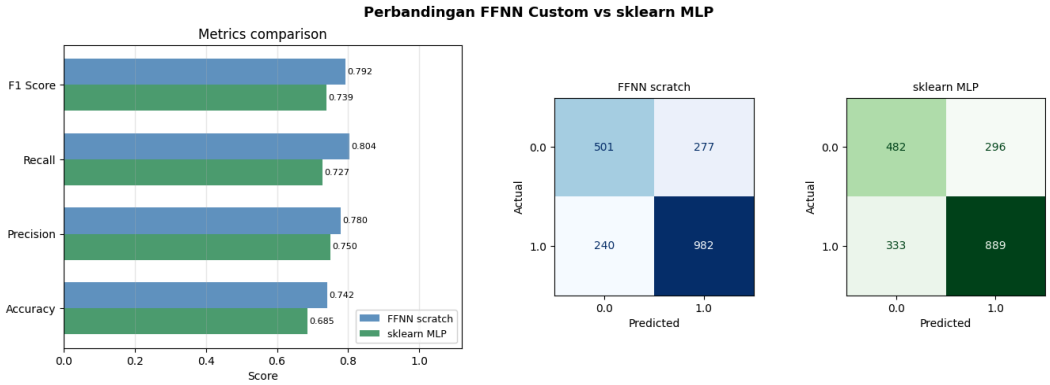
2.2.6 Perbandingan dengan Library Scikit-Learn

Notebook Pengujian: sklearn_comparison.ipynb

No	Hasil Pengujian

1	Hyperparameter - Scratch: HIDDEN_DIM = [256, 128, 64] EPOCHS = 50 BATCH_SIZE = 32 LR = 0.01 HIDDEN ACTIVATION = 'relu' SEED = 42 ALPHA = 1e-3 OUTPUT ACTIVATION = ‘sigmoid’ LOSS = ‘bce’ INIT METHOD = ‘xavier’
	Hasil: Accuracy : 0.7415 Precision : 0.7800 Recall : 0.8036 F1 Score : 0.7916
2	Hyperparameter - Sklearn MLP: hidden_layer_sizes = [256, 128, 64], activation = 'relu', solver = 'sgd', learning_rate_init = 0.01, batch_size = 32, random_state = 42, max_iter = 50, alpha = 1e-3, verbose = False
	Hasil: Accuracy : 0.6855 Precision : 0.7502 Recall : 0.7275 F1 Score : 0.7387

Penjelasan:



Gambar 2.2.6.1 Perbandingan Performa FFNN Custom vs Sklearn MLP

FFNN custom berhasil mengungguli sklearn MLP dengan selisih accuracy 5.6% pada kondisi yang identik (data, arsitektur, hyperparameter, regularisation). Keunggulan ini kemungkinan besar berasal dari efek penggunaan Xavier sebagai weight initialization yang memberikan starting point yang lebih optimal.

2.2.7 Perbandingan dengan Adam

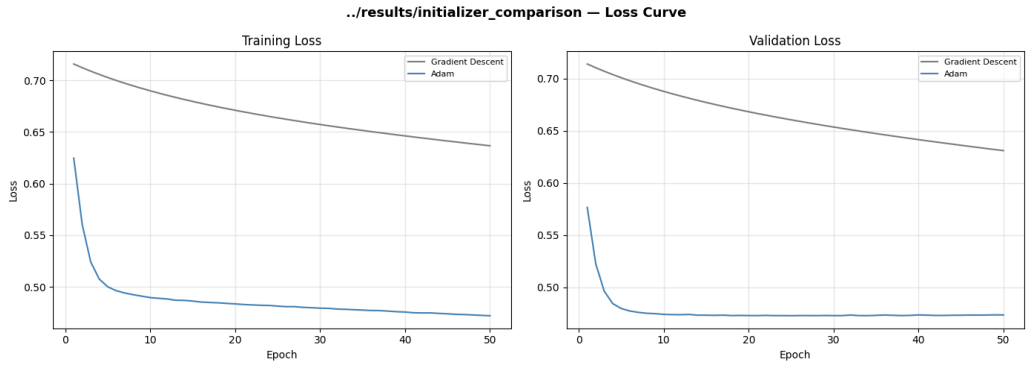
Notebook Pengujian: optimizer_comparison.ipynb

No	Hasil Pengujian
1	Hyperparameter - Gradient Descent: HIDDEN_DIM = [256, 128, 64] EPOCHS = 50 BATCH_SIZE = 32 LR = 1e-4 HIDDEN ACTIVATION = 'relu' SEED = 42 OUTPUT ACTIVATION = ‘sigmoid’ LOSS = ‘bce’ INIT METHOD = ‘xavier’ REG_TYPE = None Hyperparameter - Adam: HIDDEN_DIM = [256, 128, 64] EPOCHS = 50 BATCH_SIZE = 32

	LR = 1e-4 HIDDEN ACTIVATION = 'relu' SEED = 42 OUTPUT ACTIVATION = ‘sigmoid’ LOSS = ‘bce’ INIT METHOD = ‘xavier’ REG_TYPE = None Beta1 = 0.9 (Recommended Default) Beta2 = 0.999 (Recommended Default) Eps = 1e-8 (Recommended Default)
	Hasil: Accuracy Gradient Descent = 0.6415 Accuracy Adam = 0.7455
2	Hyperparameter - Gradient Descent: HIDDEN_DIM = [256, 128, 64] EPOCHS = 50 BATCH_SIZE = 32 LR = 1e-3 HIDDEN ACTIVATION = 'relu' SEED = 42 OUTPUT ACTIVATION = ‘sigmoid’ LOSS = ‘bce’ INIT METHOD = ‘xavier’ REG_TYPE = None Hyperparameter - Adam: HIDDEN_DIM = [256, 128, 64] EPOCHS = 50 BATCH_SIZE = 32 LR = 1e-3 HIDDEN ACTIVATION = 'relu' SEED = 42 OUTPUT ACTIVATION = ‘sigmoid’ LOSS = ‘bce’ INIT METHOD = ‘xavier’ REG_TYPE = None Beta1 = 0.9 (Recommended Default) Beta2 = 0.999 (Recommended Default) Eps = 1e-8 (Recommended Default)

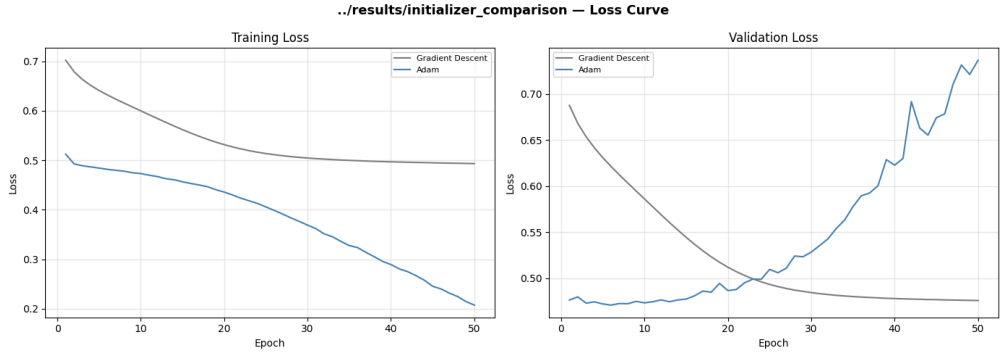
	Hasil: Accuracy Gradient Descent = 07410 Accuracy Adam = 0.6910

Penjelasan:



Gambar 2.2.7.1 Perbandingan Kurva Loss Gradient Descent vs Adam

Gradient descent memiliki konvergensi yang sangat lambat akibat learning rate yang mengecil (lr = 1e-4), berbeda dengan Adam yang berjalan dengan baik di epoch-epoch pertama sebelum kemudian melambat dan mulai plateau.



Gambar 2.2.7.2 Perbandingan Kurva Loss Gradient Descent vs Adam untuk learning rate yang lebih besar

Namun, jika learning rate dinaikkan dari 1e-4 menjadi 1e-3, gradient descent memiliki konvergensi yang sangat baik, tetapi untuk adam, validation loss malah naik. Ini kemungkinan adalah kejadian overshooting. Sehingga, untuk kecepatan konvergensi, Adam biasanya memiliki kecepatan konvergensi yang lebih baik daripada Gradient Descent, tetapi Gradient Descent memiliki konvergensi yang lebih baik dan lebih rapi.

Dan masalah lain dari Adam adalah karena kecepatan konvergensi yang lebih cepat, Adam juga lebih rentan terkena overshooting akibat adaptive scaling dan momentum. Sehingga, learning rate Adam umumnya diset lebih kecil secara numerik karena Adam sendiri menggunakan effective step size yang adaptif.

BAB III: Kesimpulan dan Saran

3.1 Kesimpulan

Berdasarkan hasil implementasi dan eksperimen yang telah dilakukan, dapat ditarik beberapa kesimpulan. Penambahan depth maupun width tidak selalu meningkatkan performa. Model dengan 2-3 *hidden layer* dan 32-128 width menunjukkan kemampuan generalisasi yang baik. Penambahan depth justru cenderung membuat overfitting lebih parah dibanding penambahan width pada dataset ini.

Perbandingan fungsi aktivasi menunjukan kemampuan yang relatif serupa. Namun, fungsi aktivasi sigmoid memiliki konvergensi paling lambat akibat vanishing gradient. Sebaliknya, aktivasi linear memiliki akurasi tinggi. Hal ini menunjukkan bahwa distribusi dataset bersifat linear. Perbandingan Learning rate didapati bahwa LR 0.1 merupakan learning rate yang paling optimal dengan hasil konvergensi yang stabil dan akurasi tinggi. Sedangkan, learning rate yang besar akan membuat overfitting, sementara nilai yang terlalu kecil akan mengakibatkan konvergensi berjalan lambat.

Penggunaan inisialisasi bobot Zero secara praktis terbukti kurang efisien karena membuat antar bobot sulit berdiferensiasi. Alternatif yang lebih baik adalah Xavier yang mampu memberikan titik awal konvergensi yang mulus dan performa akurasi yang baik. Lalu, nilai variansi sangat krusial saat menggunakan distribusi normal, dimana variansi yang terlalu tinggi dapat memicu osilasi loss yang sangat tidak stabil. Perbandingan regularisasi L1 maupun L2 tidak memproyeksikan dampak yang signifikan terhadap metrik. Hal ini disebabkan karena arsitektur dan epoch pengujian belum mendorong model ke dalam fase overfitting.

Pada perbandingan implementasi FFNN custom dengan Sklearn MLP, FFNN custom memiliki akurasi yang lebih tinggi dengan margin 5.6% walau dengan hyperparameter yang dibuat seidentik mungkin. Tetapi mungkin ada satu atau dua hal terkait parameter detail yang mempengaruhi akurasi ini, terutama untuk parameter yang ada di library, tetapi tidak ada di FFNN custom.

Terkait perbandingan dengan Adam, seperti yang dijelaskan di bagian pengujian, kecepatan konvergensi Adam lebih baik daripada Gradient Descent,

tetapi trade-offnya adalah Adam akan lebih rentan mengalami kenaikan val loss (overshooting) jika learning rate yang cukup tinggi bagi Adam walaupun itu sudah kecil bagi Gradient Descent. Dan Gradient Descent memiliki konvergensi yang lebih baik, walaupun lebih lambat.

3.2 Saran

1. Memperbanyak eksperimen untuk variasi hyperparameter agar mendapatkan hasil yang terbaik.
2. Memperbanyak eksplorasi metode preprocessing data.
3. Implementasi Dropout.

LAMPIRAN

Link Repository: https://github.com/ivant8k/FFNN-Scratch

Tabel 4.1 Pembagian tugas anggota

NIM	Pembagian Tugas
13523129	Implementasi fungsi aktivasi dan loss, Implementasi L1/L2 Regularization, Implementasi visualisasi plotting atribut model, Notebook analisis perbandingan tiap weight initializer dan perbandingan dengan library Sklearn MLP. Implementasi Adam Optimizer dan notebook analysis optimizer comparison, mengisi laporan BAB I dan BAB II (2.1.2, 2.1.3 dan hasil pengujian)
13523133	Implementasi preprocess data, implementasi initialization weight, Implementasi main.py dan notebook testing. mengerjakan analisis learning rate, dan regularisasi dari model, mengisi laporan Bab II dan III.
13523164	Implementasi FFNN model, fungsi *helper* untuk visualisasi (*training loss, validation loss*, plot distribusi model), mengisi laporan BAB II dan membuat UML Kelas Diagram, mengerjakan analisis fungsi aktivasi, dan analisis *depth* dan *weight* dari model.

REFERENSI

Stanford University. (n.d.). Neural networks part 3: Learning and evaluation. CS231n Convolutional Neural Networks for Visual Recognition. https://cs231n.github.io/neural-networks-3/#baby

Radečić, D. (2025). Softplus: The smooth activation function worth knowing. DataCamp. Softplus: The Smooth Activation Function Worth Knowing | DataCamp

Pedregosa, F., et al. (n.d.). MLPClassifier. Scikit-learn documentation. MLPClassifier — scikit-learn 1.8.0 documentation

Bendersky, E. (2016). *The softmax function and its derivative*. The Softmax function and its derivative - Eli Bendersky's website

Built In. (n.d.). *L2 regularization explained*. L1 and L2 Regularization Methods, Explained | Built In

GeeksforGeeks. (2025). Leaky ReLU activation function in deep learning. Leaky Relu Activation Function in Deep Learning - GeeksforGeeks

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. arXiv. https://arxiv.org/pdf/1412.6980

Eremenko, K. (n.d.). Deep Learning A-Z™: Artificial Neural Networks (ANN) – Module 1. SlideShare. https://www.slideshare.net/slideshow/deep-learning-az-artificial-neural-networks-ann-module-1/110292185

Lampiran Form Penggunaan AI

Dengan ini, kami:

Nama/NIM	: Ivant/13523129, Rafa/13523133, Rizain/13523164
Fakultas/Sekolah	: Sekolah Teknik Elektro dan Informatika - Komputasi
Kode Mata Kuliah	: IF3270
Nama Mata Kuliah	: Pembelajaran Mesin
Judul Tugas/Proposal	: Feedforward Neural Network

Menyatakan bahwa kami menggunakan AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang kami gunakan adalah:

Lingkup Pekerjaan	**Digunakan?**	**Tingkat Penggunaan**
Pemeriksaan Ejaan	Tidak	-
Pembuatan Teks	Tidak	-
Bantuan Diskusi dalam Penyusunan Konten	Ya	Sedang
Pembuatan Gambar, Video, dan/atau Grafik	Tidak	-
Penasehat terkait alur skema proses implementasi *source code*	Ya	Sedang

Kami juga menyatakan bahwa setiap penggunaan AI/Kecerdasan buatan yang dilakukan pada matakuliah tersebut telah dinyatakan di dalam surat ini.

Jatinangor, 16 Maret 2026	Jakarta, 16 Maret 2026	Jakarta, 16 Maret 2026
Ivant Samuel Silaban 13523129	Rafa Abdussalam Danadyaksa 13523133	Muhammad Rizain Firdaus 13523164